

Introduction à la Pagination (Jour 5)

La pagination est l'un des concepts les plus cruciaux et les plus complexes du développement d'un OS. C'est elle qui permet la gestion moderne de la mémoire et la sécurité entre processus.

1. Adresse Physique vs Adresse Virtuelle

Sans pagination, le processeur utilise des **adresses physiques** : quand vous demandez l'adresse 0x1234, vous tapez directement dans la puce de RAM à cet emplacement.

Avec la pagination, on introduit les **adresses virtuelles** :

- **Adresse Virtuelle** : Celle manipulée par les programmes (le code).
- **MMU (Memory Management Unit)** : Un composant du processeur qui traduit instantanément l'adresse virtuelle en adresse physique.
- **Adresse Physique** : L'emplacement réel dans la puce RAM.

TIP

Cela permet à deux programmes différents d'utiliser tous les deux l'adresse 0x400000, mais d'être redirigés vers des zones de RAM physique totalement différentes !

2. La Structure des Pages (x86 - 32 bits)

En mode 32 bits standard, la mémoire est découpée en blocs de **4 Ko** appelés "Pages". La traduction se fait via deux niveaux de tables :

A. Le Page Directory (Répertoire de Pages)

- Il contient 1024 entrées (**PDE** - Page Directory Entries).
- Chaque entrée pointe vers une **Page Table**.
- Un Page Directory couvre donc 4 Go de mémoire (1024 tables * 4 Mo par table).

B. Les Page Tables (Tables de Pages)

- Chaque table contient 1024 entrées (**PTE** - Page Table Entries).
- Chaque entrée pointe vers une **Page de 4 Ko** en mémoire physique.

C. Structure d'une Entrée (PDE/PTE)

Chaque entrée est un entier de 32 bits :

- **Bits 31-12** : L'adresse de la table ou de la page (alignée sur 4 Ko).
- **Bit 0 (P)** : Present (1 = en mémoire, 0 = déclenche un *Page Fault*).
- **Bit 1 (RW)** : Read/Write (0 = Lecture seule, 1 = Lecture/Écriture).
- **Bit 2 (US)** : User/Supervisor (0 = Noyau seul, 1 = Accès autorisé aux programmes).

3. Activation du mécanisme

Pour activer la pagination, il y a 3 étapes matérielles :

1. **CR3 (Contrôle Register 3)** : On doit y stocker l'adresse physique du *Page Directory*.
2. **CR0** : On doit mettre à 1 le bit **PG** (bit 31) pour dire au CPU "À partir de maintenant, traduit tout !".
3. **Identity Mapping** : Avant d'activer, on doit s'assurer que le code qui s'exécute actuellement est mappé sur la même adresse virtuelle que physique (ex: l'adresse 0x100000 virtuelle doit pointer sur 0x100000 physique). Sinon, dès l'activation, le CPU ne saura plus où se trouve l'instruction suivante et va crasher (Triple Fault).

4. Pourquoi est-ce vital pour la sécurité ?

1. **Isolation** : Un programme A ne possède dans ses tables de pages que ses propres adresses. Il lui est physiquement impossible d'écrire dans la mémoire d'un programme B car ses tables ne "connaissent" pas ces adresses.
2. **Droits d'accès** : On peut marquer des zones de mémoire comme "Lecture Seule". Si un hacker tente d'écraser le code du noyau via un buffer overflow, le CPU déclenchera un *Page Fault* immédiatement.
3. **Espace noyau protégé** : Le bit **US** empêche un programme utilisateur d'accéder aux données sensibles du système, même s'il connaît l'adresse.

5. Ce que nous allons faire cet après-midi

1. Créer un Page Directory global.
2. Créer une Page Table qui mappe les 4 premiers Mo de la RAM (Identity Mapping). Cela couvre notre Kernel et la mémoire vidéo.
3. Passer l'adresse au registre CR3 via un petit bout d'assembleur.
4. Activer le bit PG dans CR0.

```

#include "io.h"
#include "framebuffer.h"
#include "gdt.h"
#include "idt.h"
#include "paging.h"

void memory_test() {
    fb_print("    Tentative d'ecriture a 0x500000 (hors mapping)... ", 15);
    unsigned int *ptr = (unsigned int*)0x500000;
    *ptr = 0x42; // Si ca crashe ici, c'est que la pagination marche !
    fb_print("REUSSITE.\n", 10); // Vert
}

void kmain(unsigned int magic, unsigned int addr) {
    // 1. Initialisations
    fb_clear();
    gdt_init();
    idt_init();
    pic_remap();

    fb_print("--- TEST DE PAGINATION ---\n\n", 11);

    fb_print("1. TEST SANS PAGINATION :\n", 14);
    memory_test();

    fb_print("\n2. ACTIVATION DE LA PAGINATION...\n", 14);
    paging_init();
    fb_print("Pagination active (Identity mapping 4Mo OK).\n", 10);

    fb_print("\n3. TEST AVEC PAGINATION :\n", 14);
    memory_test();

    while(1);
}

```

le test se deroulera en 3 phases :

Étape 1 : Le test SANS pagination

- **Est-ce que ça crache ? : NON.**
- **Pourquoi ?** : À ce stade, la pagination est désactivée (le bit PG de CR0 est à 0). Le processeur travaille avec des **adresses physiques**. Comme tu as donné 32 Mo de RAM à QEMU, l'adresse
 - 0x500000
 - (5 Mo) existe réellement dans la puce de RAM. Le processeur écrit
 - 0x42
 - à cet endroit sans aucun problème.

Étape 2 : L'activation de la pagination

- **Action** : On charge le Page Directory qui dit au CPU : "Désormais, ne regarde plus la RAM directement. Utilise mes tables."
- **Configuration** : Tes tables ne mappent que les **4 premiers Mo** (de
 - 0x0
 - à
 - 0x3FFFFFF
 -).
- **Résultat** : Tout ce qui est au-dessus de 4 Mo (comme l'adresse
 - 0x500000
 -) devient "invisible" et "interdit" pour le processeur.

Étape 3 : Le test AVEC pagination

- **Est-ce que ça crache ? : OUI.**
- **Pourquoi précisément à ce moment ?** :
 1. Le code tente d'écrire à
 2. 0x500000
 3. .
 4. Le processeur (la MMU) vérifie dans tes tables de pages si l'adresse
 5. 0x500000
 6. est "Présente".
 7. Elle voit que le bit **Present** est à **0** pour cette zone (car on ne l'a pas mappée).
 8. Le CPU génère immédiatement une **Exception 14 (Page Fault)**. C'est une alerte de sécurité matérielle.

Ma preuve est :

```
one@DESKTOP-5RV2RM1: /mr  one@DESKTOP-5RV2RM1: /m  Windows PowerShell
NASM: io.s -> io.o
NASM: loader.s -> loader.o
NASM: paging_asm.s -> paging_asm.o
--- Liaison du Kernel ---
ld: warning: paging_asm.o: missing .note.GNU-stack section implies executable stack
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
--- Creation de l'ISO ---
xorriso 1.5.6 : RockRidge
Drive current: -outdev 'stdio:os.iso'
Media current: stdio file,
Media status : is blank
Media summary: 0 sessions,
Added to ISO image: direct
xorriso : UPDATE : 588
Added to ISO image: direct
xorriso : UPDATE : 593
xorriso : NOTE : Copying t
xorriso : UPDATE : 87.59%
ISO image produced: 5809 s
Written to medium : 5809 s
Writing to 'stdio:os.iso' completed successfully.

--- Terminé avec succès ! ---
one@DESKTOP-5RV2RM1:/mnt/c/Users/MOH/Documents/Lab1_CYBER/5_ProjetOS$ qemu-system-i386 -cdrom os.iso -serial stdio -boot d
```

```
one@DESKTOP-5RV2RM1: /mr  one@DESKTOP-5RV2RM1: /m  Windows PowerShell
NASM: io.s -> io.o
NASM: loader.s -> loader.o
NASM: paging_asm.s -> paging_asm.o
--- Liaison du Kernel ---
ld: warning: paging_asm.o: missing .note.GNU-stack section implies executable stack
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
--- Creation de l'ISO ---
xorriso 1.5.6 : RockRidge
Drive current: -outdev 'stdio:os.iso'
Media current: stdio file,
Media status : is blank
Media summary: 0 sessions,
Added to ISO image: direct
xorriso : UPDATE : 588
Added to ISO image: direct
xorriso : UPDATE : 593
xorriso : NOTE : Copying t
xorriso : UPDATE : 87.59%
ISO image produced: 5809 s
Written to medium : 5809 s
Writing to 'stdio:os.iso' completed successfully.

--- Terminé avec succès ! ---
one@DESKTOP-5RV2RM1:/mnt/c/Users/MOH/Documents/Lab1_CYBER/5_ProjetOS$ qemu-system-i386 -cdrom os.iso -serial stdio -boot d
```

Vous pouvez voir que la vm reboot immédiatement pour éviter l'écriture.

Le mécanisme de reboot est donc une bonne chose du point de vue cyber mais entraîne un autre problème comme les DOS et empêche la mise en place des logs analysables. Pour éviter le crash, on peut faire une interruption comme on l'a vu dans les tâches d'hier en utilisant un `idt_gate` pour la gestion du crash vm redémarrer tout le temps.

On rétablit l'interruption en la définissant dans les fichiers `interruption.s`, et on définit son handler dans `idt.c`

```
one@DESKTOP-5RV2RM1: /mr  one@DESKTOP-5RV2RM1: /m  Windows PowerShell  X + v -  X
```

```
NASM: io.s -> io.o
NASM: loader.s -> loader.o
NASM: paging_asm.s -> paging_asm.o
--- Liaison du Kernel ---
ld: warning: paging_asm.o: missing .note.GNU-stack: beware of using stack cookies [skipped]
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
--- Creation de l'ISO ---
xorriso 1.5.6 : RockRidge filesystem manipulation
Drive current: -outdev 'stdio:os.iso'
Media current: stdio file, overwriteable
Media status : is blank
Media summary: 0 sessions, 0 data blocks, 0 data files, 0 volumes
Added to ISO image: directory '/='/tmp/grub
xorriso : UPDATE : 588 files added in
Added to ISO image: directory '/='/mnt/c/Users/MOH/Documents/Lab1_CYBER/5_ProjetOS
xorriso : UPDATE : 593 files added in
xorriso : NOTE : Copying to System Area: 512 bytes from file '/usr/lib/grub/i386-pc/boot_hybrid.img'
xorriso : UPDATE : 56.74% done
ISO image produced: 5809 sectors
Written to medium : 5809 sectors at LBA 0
Writing to 'stdio:os.iso' completed successfully.

--- Terminé avec succès ! ---
one@DESKTOP-5RV2RM1:/mnt/c/Users/MOH/Documents/Lab1_CYBER/5_ProjetOS$ qemu-system-i386 -cdrom os.iso -serial stdio -boot d
```

QEMU - Press Ctrl+Alt+G to release grab

Machine View

```
--- TEST DE PAGINATION ---
1. TEST SANS PAGINATION :
Tentative d'écriture à 0x500000 (hors mapping)... REUSITE.
2. ACTIVATION DE LA PAGINATION...
Pagination active (identity mapping 4Mo OK).
3. TEST AVEC PAGINATION :
Tentative d'écriture à 0x500000 (hors mapping)...
>>> PAGE FAULT DETECTE ! (Accès mémoire interdite) <<<
```

```
one@DESKTOP-5RV2RM1: /mr  one@DESKTOP-5RV2RM1: /m  Windows PowerShell  X + v -  X
```

```
NASM: io.s -> io.o
NASM: loader.s -> loader.o
NASM: paging_asm.s -> paging_asm.o
--- Liaison du Kernel ---
ld: warning: paging_asm.o: missing .note.GNU-stack: beware of using stack cookies [skipped]
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
--- Creation de l'ISO ---
xorriso 1.5.6 : RockRidge filesystem manipulation
Drive current: -outdev 'stdio:os.iso'
Media current: stdio file, overwriteable
Media status : is blank
Media summary: 0 sessions, 0 data blocks, 0 data files, 0 volumes
Added to ISO image: directory '/='/tmp/grub
xorriso : UPDATE : 588 files added in
Added to ISO image: directory '/='/mnt/c/Users/MOH/Documents/Lab1_CYBER/5_ProjetOS
xorriso : UPDATE : 593 files added in
xorriso : NOTE : Copying to System Area: 512 bytes from file '/usr/lib/grub/i386-pc/boot_hybrid.img'
xorriso : UPDATE : 56.74% done
ISO image produced: 5809 sectors
Written to medium : 5809 sectors at LBA 0
Writing to 'stdio:os.iso' completed successfully.

--- Terminé avec succès ! ---
one@DESKTOP-5RV2RM1:/mnt/c/Users/MOH/Documents/Lab1_CYBER/5_ProjetOS$ qemu-system-i386 -cdrom os.iso -serial stdio -boot d
```

QEMU - Press Ctrl+Alt+G to release grab

Machine View

```
--- TEST DE PAGINATION ---
1. TEST SANS PAGINATION :
Tentative d'écriture à 0x500000 (hors mapping)... REUSITE.
2. ACTIVATION DE LA PAGINATION...
Pagination active (identity mapping 4Mo OK).
3. TEST AVEC PAGINATION :
Tentative d'écriture à 0x500000 (hors mapping)...
>>> PAGE FAULT DETECTE ! (Accès mémoire interdite) <<<
```

